

Phase -1.10 — Sequence Diagrams & Business Workflows

Objective

The High-Level Design defines the system architecture.

The ER Diagram defines how data is stored.

The API Contract defines how clients communicate with the application.

This phase answers another critical question:

What exactly happens inside the system after an API request is received?

Sequence diagrams describe the chronological interaction between different components of the system.

They help developers understand:

- Request flow
- Component responsibilities
- Order of execution
- Database interactions
- Cache interactions
- Background jobs
- Error handling
- Transaction boundaries

These diagrams become invaluable when debugging, implementing features, or explaining the system during interviews.

Why Sequence Diagrams?

Sequence diagrams answer questions such as:

- Which component receives the request first?
- When does validation occur?
- When is the database accessed?
- When is Redis used?
- When are background jobs created?
- When should a transaction begin?
- When should a transaction commit or rollback?

Without sequence diagrams, developers often implement different flows for similar features, leading to inconsistent architecture.

Components Used in the Diagrams

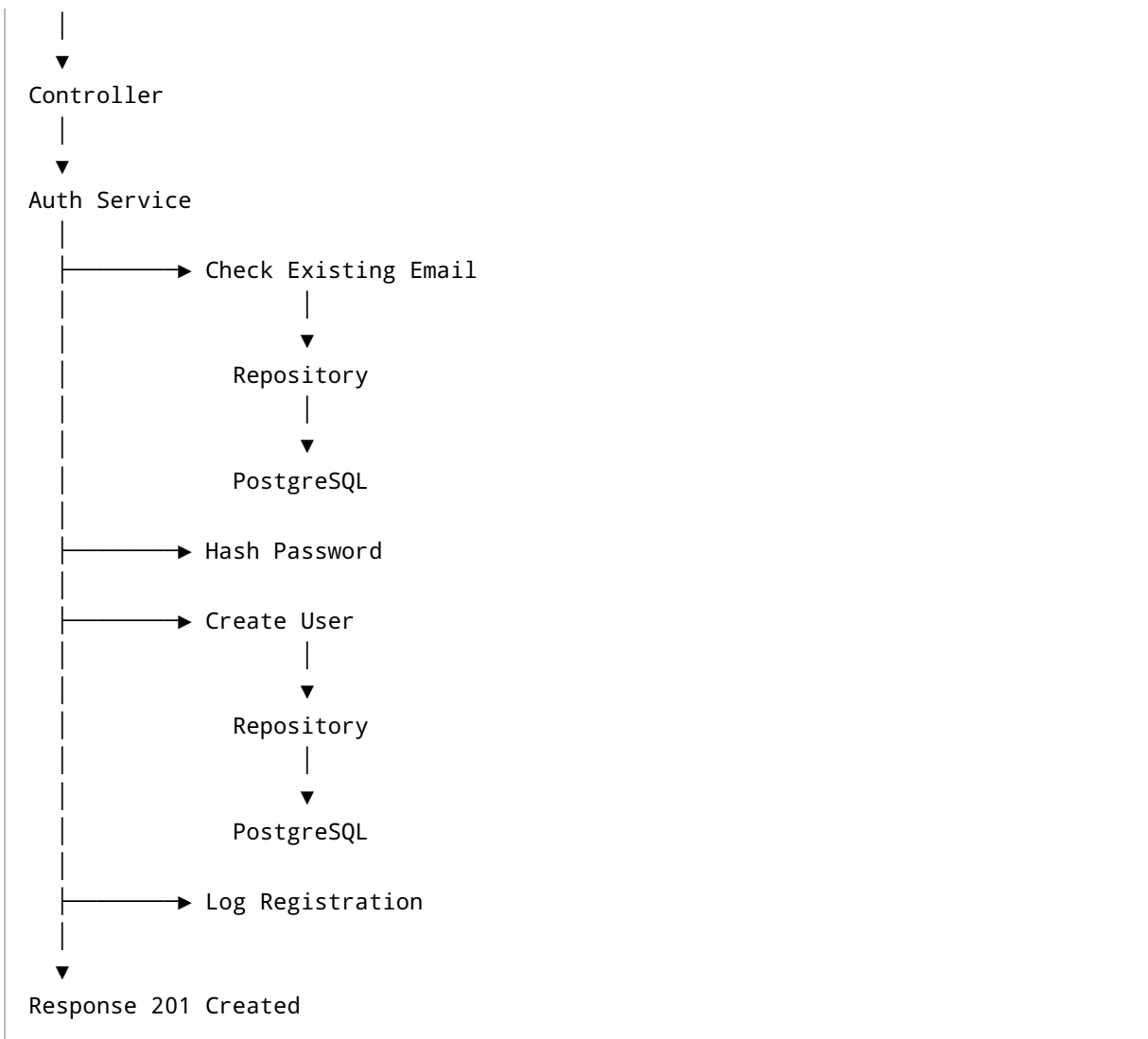
Throughout this document, the following participants appear repeatedly.

```
Client
Router
Middleware
Controller
Service
Repository
PostgreSQL
Redis
BullMQ
Worker
Logger
```

Every business workflow is composed of interactions between these components.

Workflow 1 — User Registration

```
Client
|
| POST /auth/register
▼
Router
|
▼
Validation Middleware
```



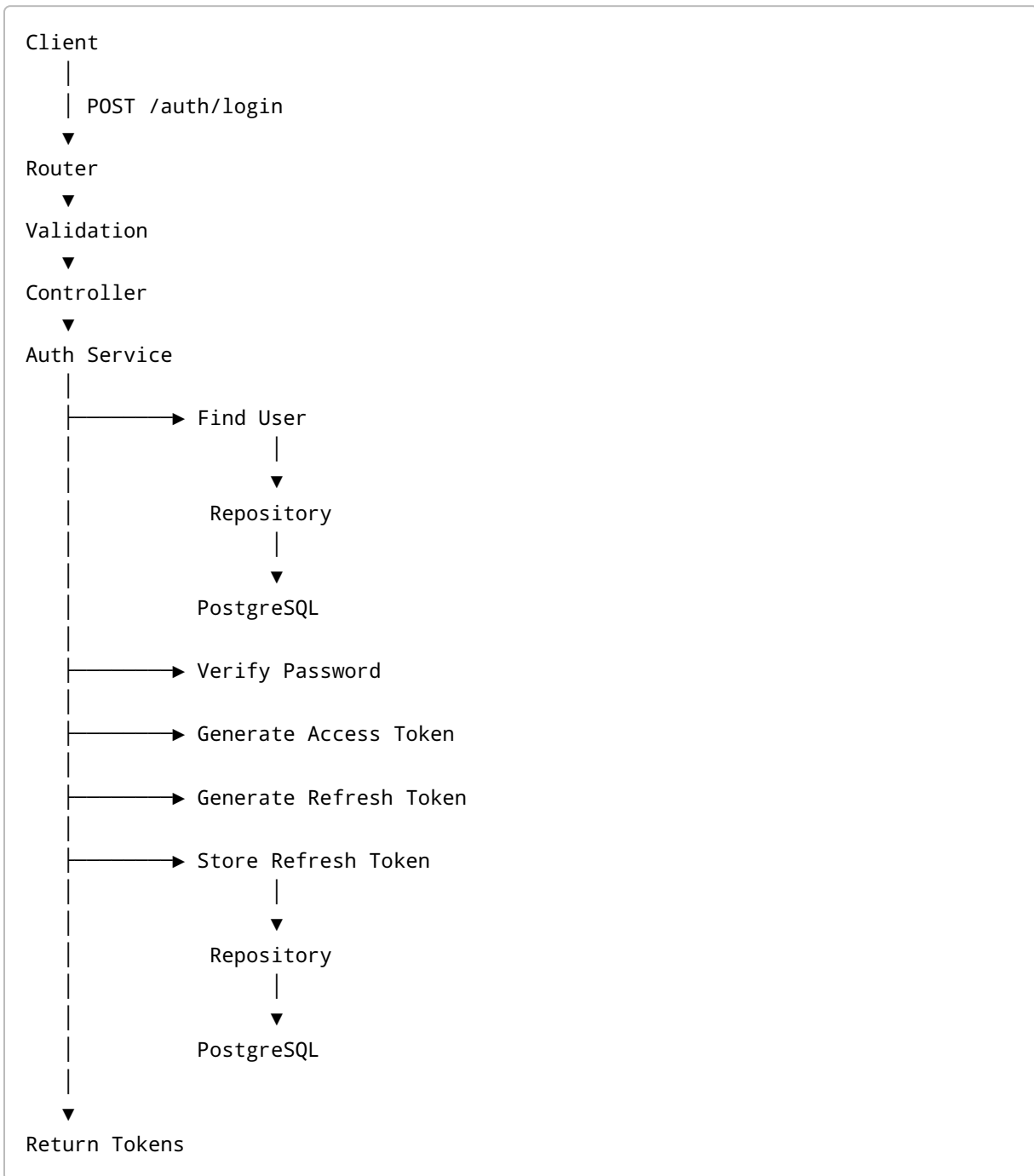
Important Notes

No JWT is generated during registration.

The user must authenticate separately.

Passwords are hashed before reaching the database.

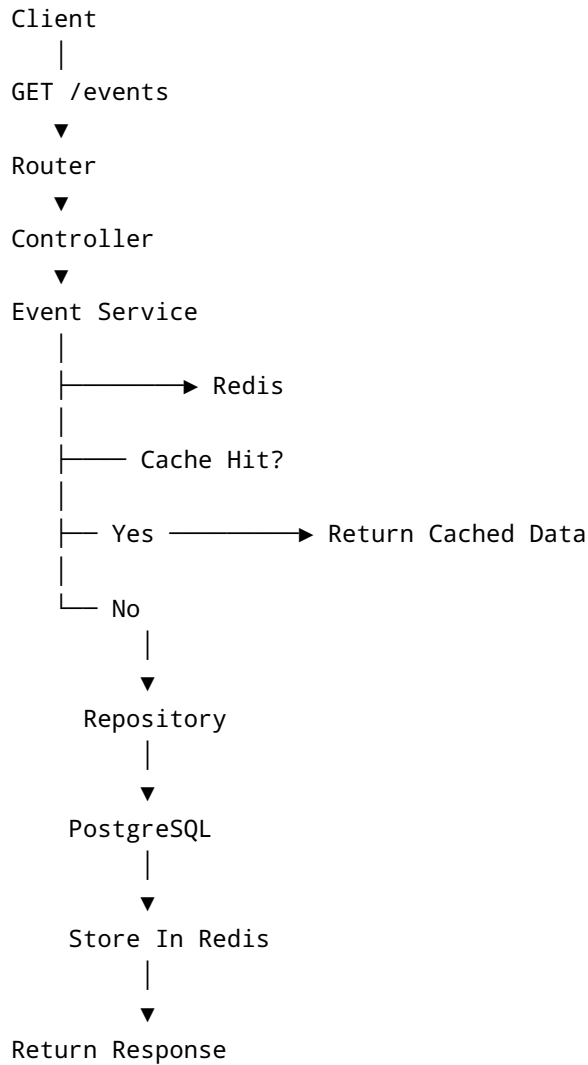
Workflow 2 — User Login



Transaction Boundary

No database transaction is required because only one write operation occurs.

Workflow 3 — Browse Events



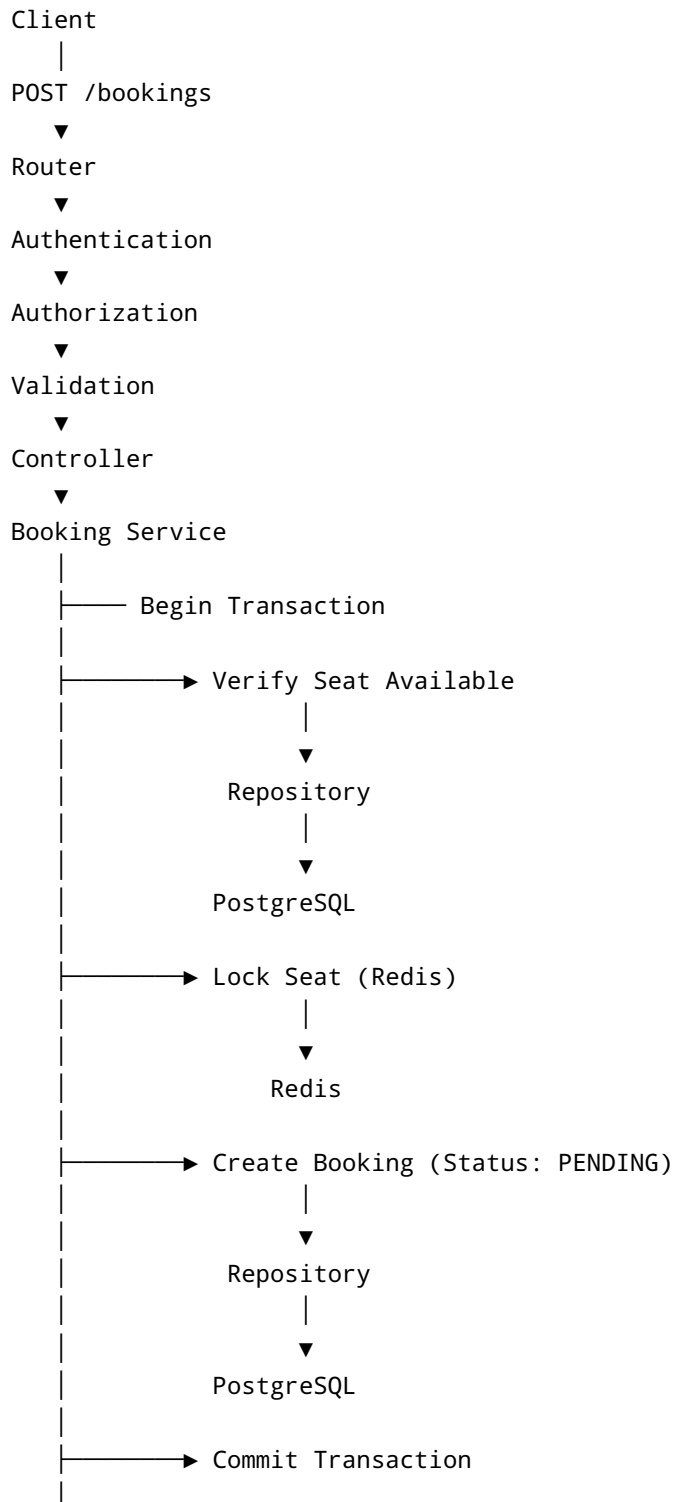
Learning Objective

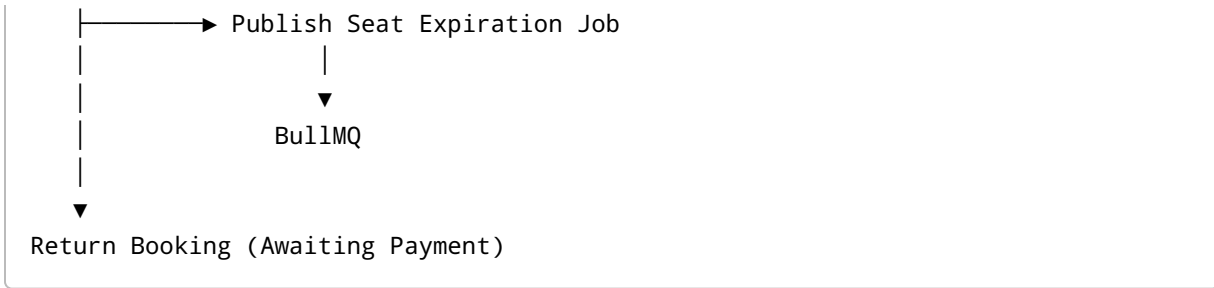
This is an example of the **Cache-Aside Pattern**, one of the most common caching strategies in backend systems.

When a new data is created by the admin the cache is dismissed and hits database for response then the data gets cached again.

Workflow 4 — Create Booking

This is the most important workflow in the entire application.





Important Clarification

The booking creation flow is intentionally split into two phases:

Phase 1 — Reservation (Synchronous)

- Seat availability is checked from the database.
- A temporary lock is placed in Redis to prevent concurrent bookings.
- A booking is created with status `PENDING`.
- The transaction is committed.

Phase 2 — Payment (Separate Workflow)

- Payment is processed in a separate request.
- Only after successful payment:
- Seat status is updated to `BOOKED` in PostgreSQL.
- Redis lock is released.
- Booking status is updated to `CONFIRMED`.

Failure Handling

- If payment fails or is not completed within the allowed time:
- BullMQ background job triggers seat release.
- Redis lock is removed.
- Booking status is updated to `CANCELLED`.
- Seat remains available in the database.

Key Design Principle

Payment is **not included inside the booking transaction**.

This ensures:

- Faster response time for booking creation.
- Better scalability.
- Reduced transaction duration.
- Avoidance of long-running database locks.

This design follows industry-standard patterns used in high-scale booking systems.

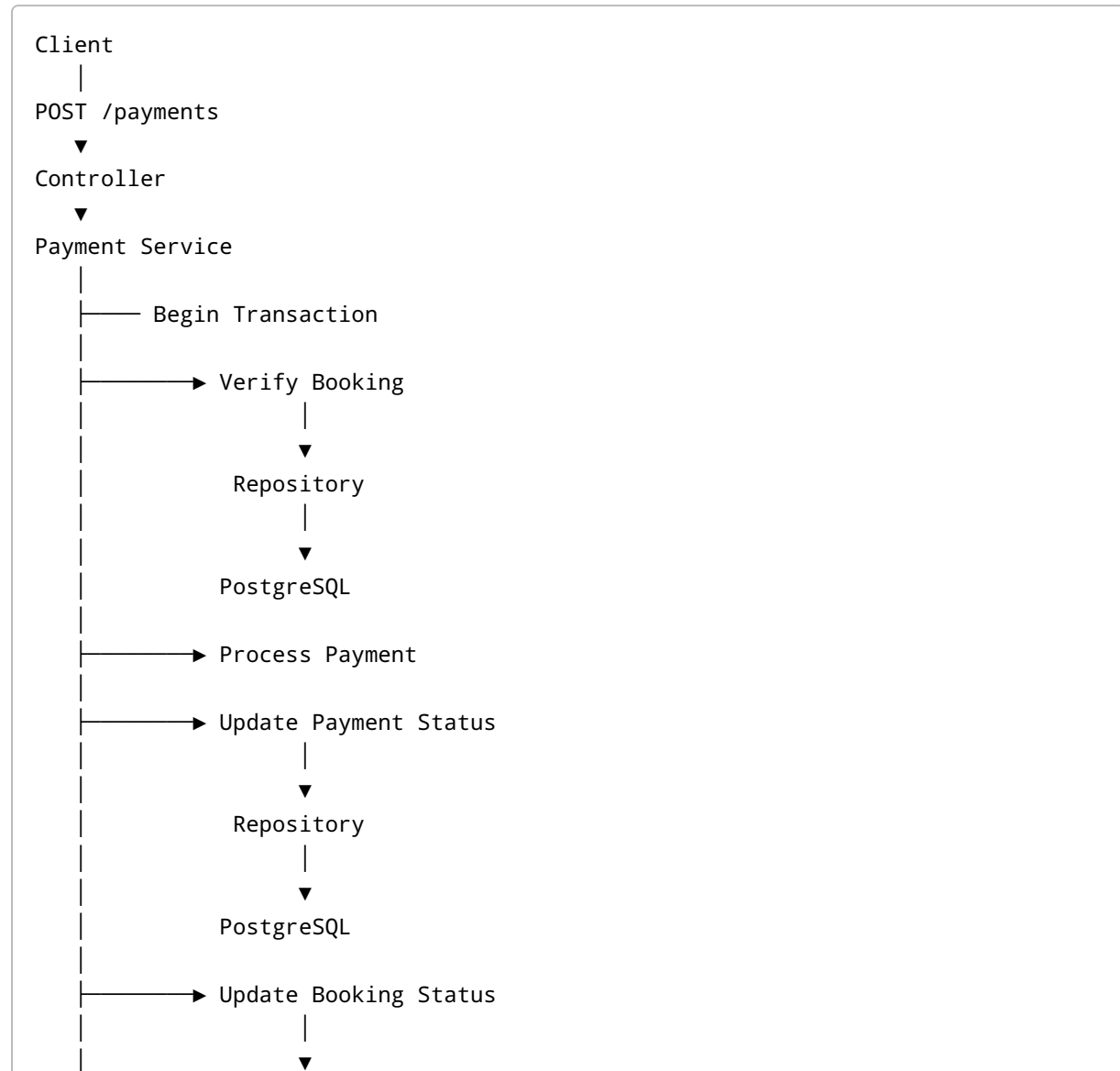
Why a Transaction?

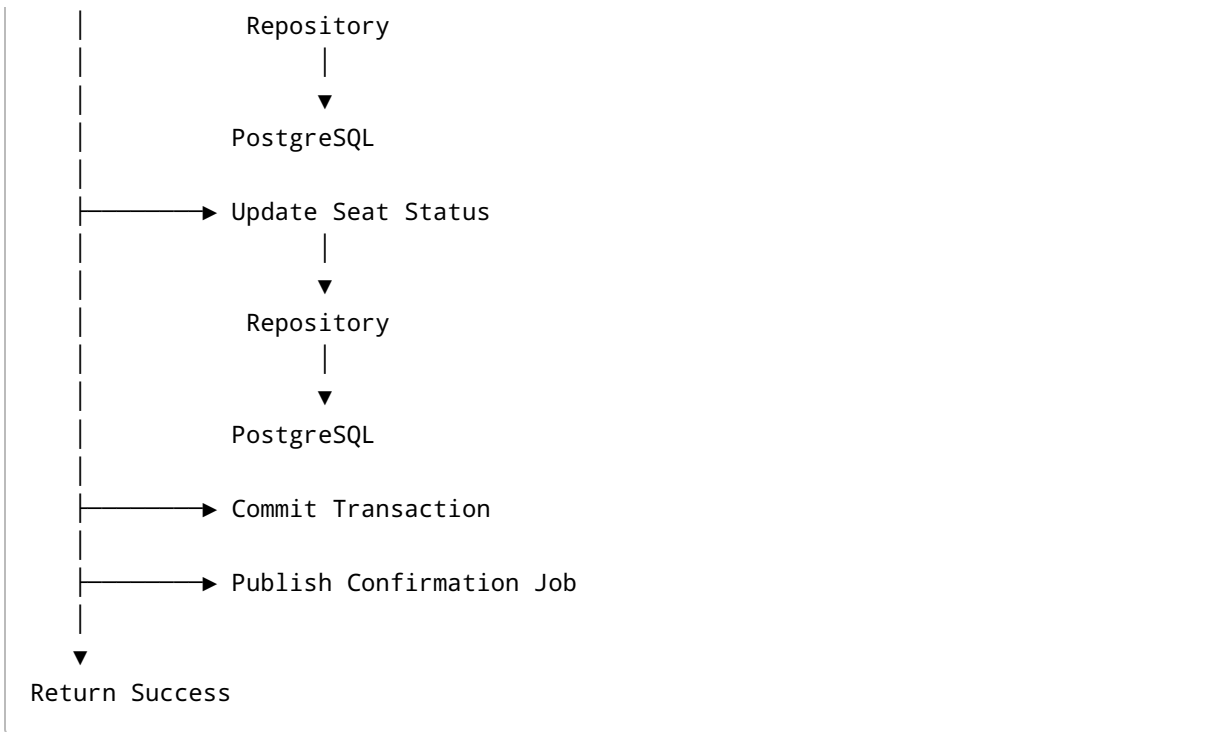
The following operations must either succeed together or fail together.

- Seat verification
- Booking creation
- Seat update

If any operation fails, everything must roll back.

Workflow 5 — Successful Payment



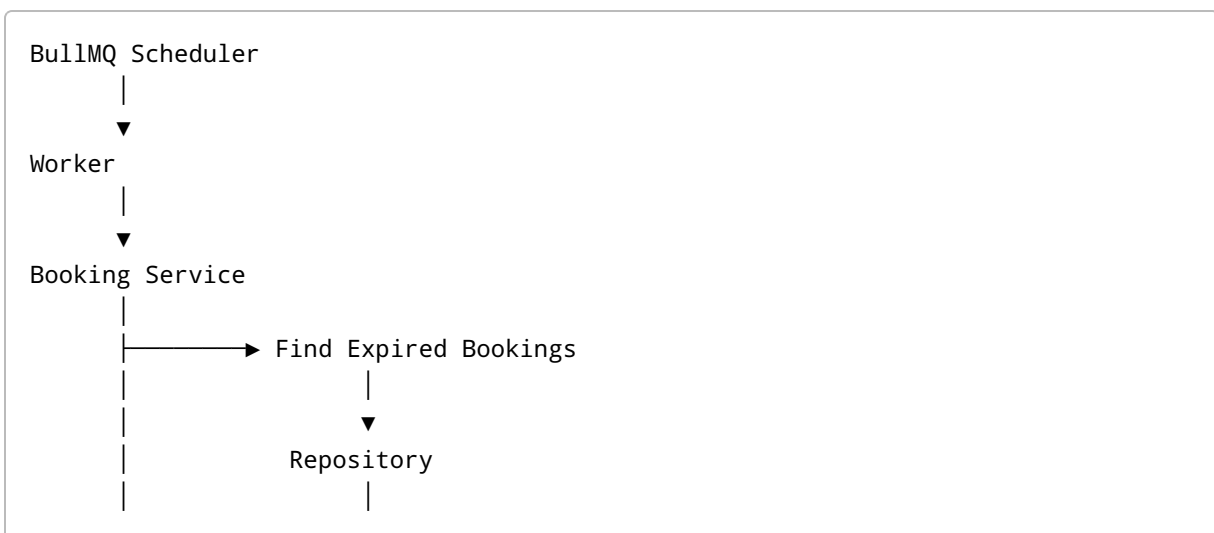


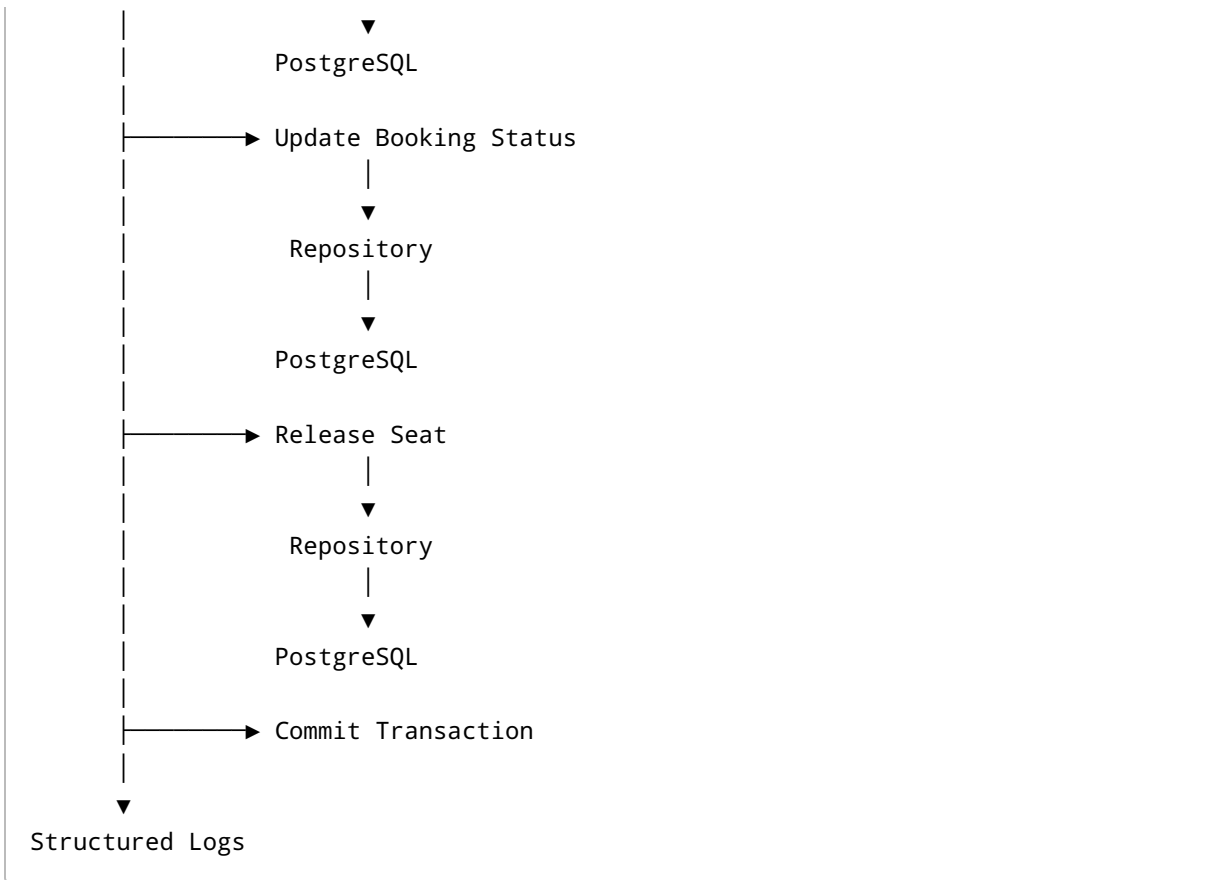
Background Work

The confirmation notification is intentionally asynchronous.

The client should never wait for notification processing.

Workflow 6 — Seat Hold Expiration

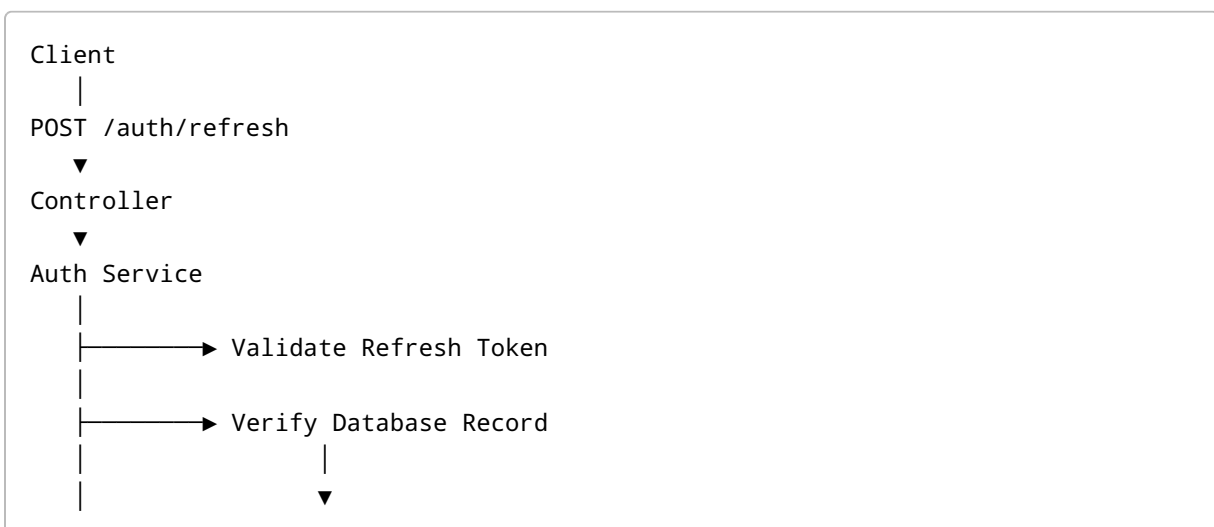


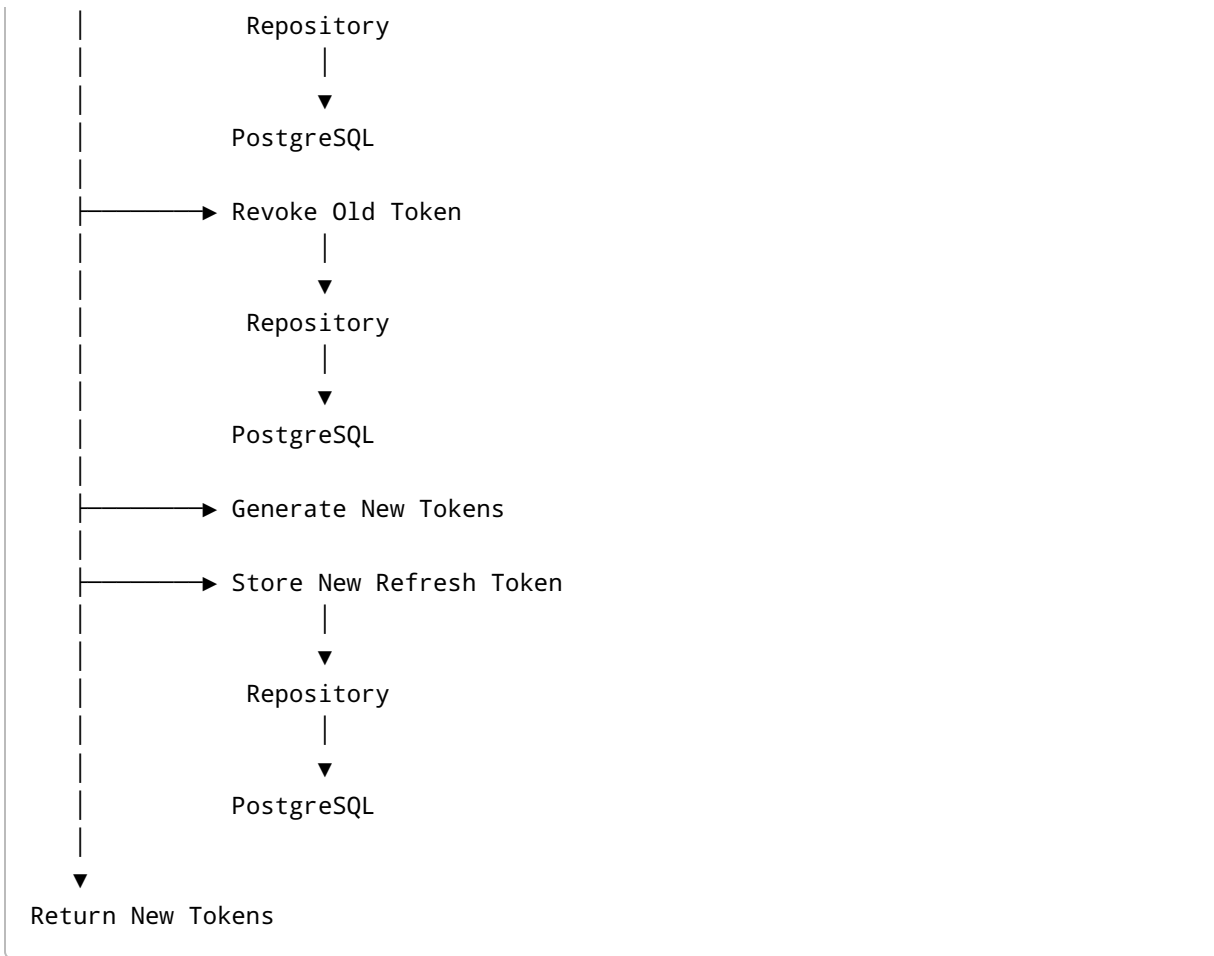


No HTTP request is involved.

This workflow is entirely system initiated.

Workflow 7 — Refresh Token Rotation





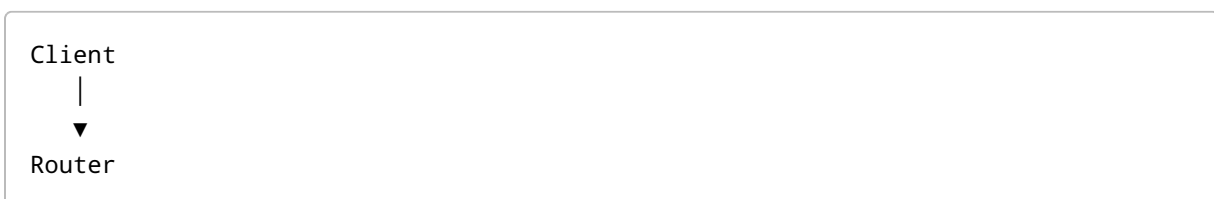
Security Principle

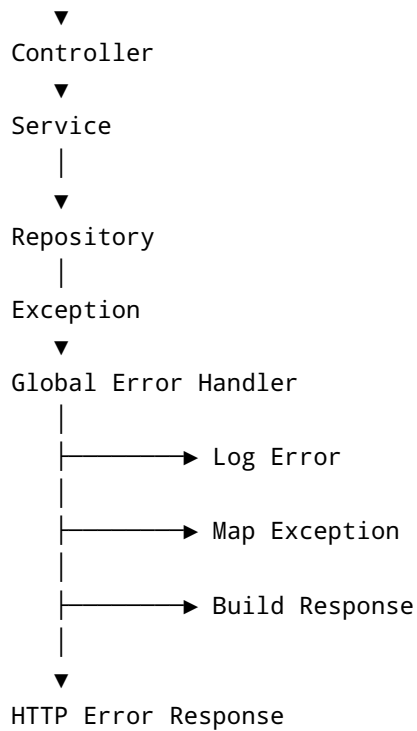
Every refresh operation invalidates the previous refresh token.

Instead of revoking have a boolean status `isExpired`, just like `soft delete`. A background job can then take care of it as a cron job in the background for deleting.

This significantly reduces replay attack risks.

Workflow 8 — Global Error Handling

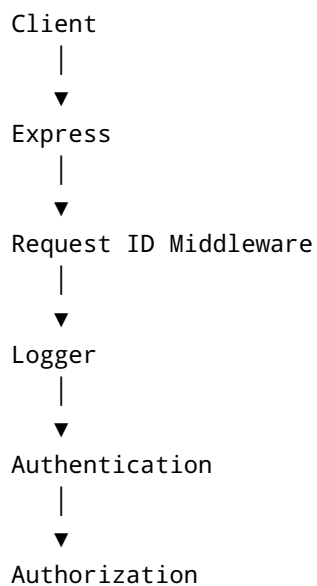




Controllers should never manually catch and format business exceptions.

Workflow 9 — Request Lifecycle

This diagram summarizes every incoming request.





Every API should follow this lifecycle unless there is a compelling architectural reason not to.

Transaction Guidelines

Not every request requires a database transaction.

Transactions **should** be used for:

- Booking creation
- Payment processing
- Seat release
- Seat reservation
- Token rotation
- Multi-table updates

Transactions **should not** be used for:

- Login reads
- Event browsing
- Viewing bookings
- Health checks
- Metrics endpoints

Using unnecessary transactions reduces scalability.

Synchronous vs Asynchronous Work

Synchronous

The client waits for completion.

Examples:

- Login
 - Registration
 - Booking creation
 - Payment
-

Asynchronous

The client receives a response immediately.

Processing continues in the background.

Examples:

- Seat expiration
 - Notifications
 - Cleanup jobs
 - Retry jobs
 - Analytics
 - Audit logging (future)
-

Deliverables

At the completion of this phase, the runtime behavior of the application is completely documented.

Every important workflow now defines:

- Request sequence.
- Component interactions.
- Transaction boundaries.
- Background processing.
- Cache usage.
- Database access.
- Error propagation.
- Response generation.

These sequence diagrams become the implementation blueprint for every feature developed in subsequent phases.

The next phase, **Phase -1.11 — Capacity Planning & Scalability Assumptions**, will define expected traffic, storage growth, concurrency estimates, scaling strategies, bottlenecks, and infrastructure assumptions before implementation begins.